Eugene Quek's Project Portfolio Page

Project: ClubFlow

ClubFlow is a command-line application written in Java that helps student clubs manage events, members, and related information.

Given below are my contributions to the project.

- **Code contributed**: tP Code Dashboard
- **New Feature: Event Management and Partial Editing** (PR #24, PR #46)
  - **What it does:** Connects event creation, viewing, and deletion to shared club data, and allows users to edit selected event details without re-entering the remaining information.
  - **Justification:** Supports the event planning workflow as dates, budgets, venues, and logistics change.
  - **Highlights:** Implemented shared event storage and reusable argument validation. Editing validates the request before replacing the event, preserving omitted details, completion state, and list position. Added labelled event details and consistent handling of duplicate or missing event tags.
- **New Feature: Event Search with Combined Filters** (PR #57)
  - **What it does:** Provides `findEvent` (`fe`) with filters for text, event dates, planning deadlines, exact budgets, budget ranges, and completion status.
  - **Justification:** Helps users locate relevant events, such as pending events within a budget, without inspecting every event individually.
  - **Highlights:** Separated command validation from search predicates. All supplied filters must match; text matching ignores case and results retain event order. Filters are compiled once per search. Validation rejects blank values, invalid dates or budgets, and conflicting budget bounds.
- **New Feature: Event Completion Tracking** (PR #46)
  - **What it does:** Adds `mark` and `unmark`, displays completion status, and preserves that status across saving and loading.
  - **Justification:** Lets club organisers distinguish completed events from pending work.
  - **Highlights:** Implemented a shared completion command with an event-specific handler. Repeated requests retain the requested state. Extended persistence while accepting older records without a completion field, and tested the create, mark, save, reload, view, and unmark workflow.
- **Enhancements to Existing Features**:
  - Added defensive validation, assertions, and checked application errors so invalid requests report failures without stopping the command loop (PR #29, PR #46).
  - Routed diagnostics to `clubflow.log` to keep command output readable (PR #30).
  - Added JUnit coverage for event commands, validation failures, unchanged state after failed operations, read-only snapshots, malformed save records, and restart persistence.
  - Fixed help and spelling suggestions to retain registered command casing, with regression tests (commit).

- **Documentation**:
  - **User Guide:** Replaced starter content with ClubFlow instructions and examples; documented event creation, editing, viewing, deletion, completion, and combined event search.
  - **Developer Guide:** Wrote the architecture overview, common command contracts, validation, and error handling, event implementation and design decisions, logging guidance, and event manual tests.
  - **UML diagrams:** Added `Architecture`; the `CreateEventSequence`, `EditEventSequence`, `FindEventSequence`, and `CompletionSequence` diagrams; the `EventSearchClass` and `EventValidationClass` diagrams; and the `EventReplacementObject` and `CompletionWiringObject` diagrams. Sources and renders are in the diagram folders.
- **Team-based Contributions**:
  - Provided shared event storage and validation used across event commands, and aligned event errors with the application's exception handling.
  - Organised commands into feature subpackages, corrected the application entry point, and standardised the team's UML sources and SVG renders with shared styling and maintenance instructions.
  - Reviewed and merged teammates' changes, including member enhancements, multiple-club support, and Developer Guide contributions.
- **Review Contributions**:
  - Suggested code formatting improvements in PR #25 and requested corrected Developer Guide numbering in PR #59.
  - Reviewed and approved PR #52 (member enhancements), PR #54 (multiple clubs), and PR #55 (photo management).
- **Use of AI Tools**:
  - **Implementation:** Used ChatGPT to discuss implementation approaches and assist with developing and refining event features, argument validation, and error handling.
  - **Testing:** Used ChatGPT to identify edge cases and help generate and refine JUnit tests for event editing, search, completion tracking, and persistence.
  - **Documentation:** Used ChatGPT to help draft and improve User Guide and Developer Guide content, including command examples, implementation explanations, and design rationale.
  - **UML diagrams:** Used ChatGPT to help create and refine PlantUML sources for architecture, class, object, and sequence diagrams describing event workflows and shared components.
  - Used Codex to help organise and draft this portfolio using Git history, project documentation, and pull request discussions.